

□ Understanding the function and use of Tree Data Structure □

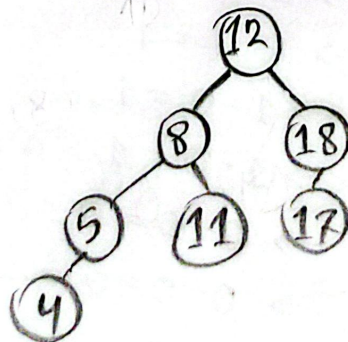
Objectives:

- To use the concepts of data structures with real life applications
- To design the particular data structures for understanding of how appropriate data structure selection improves system efficiency

Background:

A Tree is a non-linear hierarchical data structure made of nodes connected by edges. It represents the data in a parent-child relationship where the top node is called root node and the node without children is called leaf node. Tree operations include - Traversing, Searching, Insertion and Deletion

□ The given tree,



Tasks:

(i) All possible names of the tree:

- Binary Tree (Each node has max 2 child)

- Node 12 $\rightarrow$ 2 child (8 and 18)
- Node 8 $\rightarrow$ 2 child (5 and 11)
- Node 18 $\rightarrow$ 1 child (17)
- Node 5 $\rightarrow$ 1 child (4)
- Node 11 $\rightarrow$ 0 child
- Node 17 $\rightarrow$ 0 child
- Node 4 $\rightarrow$ 0 child

- Binary Search Tree (Left child $<$ Parent $<$ Right child)

- Node 12 $\rightarrow 8 < 12 < 18$
- Node 8 $\rightarrow 5 < 8 < 11$
- Node 18 $\rightarrow 17 < 18$
- Node 5 $\rightarrow 4 < 5$
- Node 4, 11, 17 $\rightarrow$ LEAF NODES (no child)

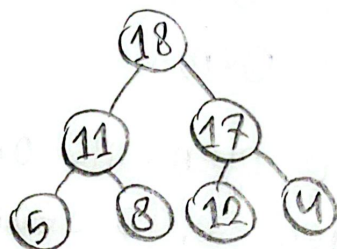
- Balanced Binary Tree ($H_{LST} - H_{RST} \leq 1$ for every node)

- Node 12 $\rightarrow 3 - 2 = 1$
- Node 8 $\rightarrow 2 - 1 = 1$
- Node 18 $\rightarrow 1 - 0 = 1$
- Node 5 $\rightarrow 1 - 0 = 1$
- Node 11 $\rightarrow 0 - 0 = 0$
- Node 17 $\rightarrow 0 - 0 = 0$
- Node 4 $\rightarrow 0 - 0 = 0$

- LST = Left Sub Tree
- RST = Right Sub Tree

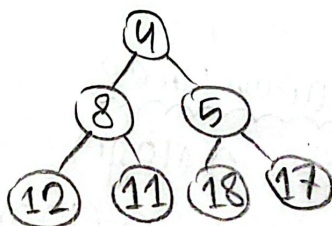
(ii) Max Heap:

A complete binary tree where the root is the maximum value



Min Heap:

A complete binary tree where the root is the minimum value



(iii) Real world Application For;

- Binary Tree → Expression evaluation in compilers
(Parsing $((a+b)*c)$)
- Binary Search Tree → Fast searching in databases
(Searching contacts in a phone-book)
- Balanced Binary Tree → self balancing databases/
indexing systems
(Automatically maintains height
balance to prevent worst case)
- Max Heap → OS scheduler
(Executes highest priority processes first)
- Min Heap → Google Map
(Finding the shortest/closest path first)

(iv) Time Complexity Analysis:

① Binary Tree

- Searching : $O(n)$
- Insertion : $O(n)$
- Deletion : $O(n)$

In Binary Tree, elements are not arranged in any particular order. So, to search for a specific value, it may be necessary to traverse all nodes in the worst case. Similarly, insertion and deletion may require traversal to locate the appropriate position, resulting in linear time complexity

② Binary Search Tree

- Best Case : $O(\log n)$
- Average Case : $O(\log n)$
- Worst Case : $O(n)$

In balanced conditions, operations are efficient due to divide-and-conquer. However, if the tree becomes skewed (like a linked list), complexity degrades to $O(n)$

③ Balanced Binary Tree

- Searching : $O(\log n)$
- Insertion : $O(\log n)$
- Deletion : $O(\log n)$

The height is always proportional to $\log n$, so all operations remain efficient. Additional rotations may be required during insertion and deletion to maintain balance

④ Max Heap

- Insertion : $O(\log n)$
- Deletion : $O(\log n)$
- Accessing maximum element: $O(1)$

Insertion and deletion require heapify operations, which take logarithmic time. However, accessing the maximum element is constant time since it is always at root

⑤ Min Heap

- Insertion : $O(\log n)$
- Deletion : $O(\log n)$
- Accessing minimum element: $O(1)$

The structure is similar to max heap, but it prioritizes the smallest element. The root always contains the minimum value, allowing constant time access

~~(v) Determining~~

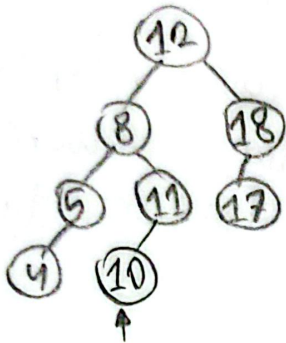
(v) Choosing the best among them depends entirely on the intended use cases of the system

- Best for simple ordered data $\rightarrow$ BST
- Best for priority handling $\rightarrow$ Max Heap
- Best for shortest path finding $\rightarrow$ Min Heap
- Best for searching $\rightarrow$ Balanced Binary Tree

However, Balanced Binary Tree is overall considered as best because it guarantees $O(\log n)$ performance in all cases.

(vi) Inserting node 10,

• Hand:



• Code:

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int data;
    Node *left;
    Node *right;

    Node(int value) {
        data = value;
        left = right = NULL;
    }
};
```

```
Node* insertIterative(Node* root, int key) {
    Node* parent = NULL;
    Node* curr = root;

    while (curr != NULL) {
        parent = curr;
        if (key < curr->data)
            curr = curr->left;
    }
```

```
else if (key > curr->data)
```

```
    curr = curr->right;
```

```
else
```

```
    return root;
```

```
}
```

```
Node* newNode = new Node(key);
```

```
if (parent == NULL)
```

```
    return newNode;
```

```
if (key < parent->data)
```

```
    parent->left = newNode;
```

```
else
```

```
    parent->right = newNode;
```

```
    return root;
```

```
}
```

```
void inorder(Node* root) {
```

```
    if (root != NULL) {
```

```
        inorder(root->left);
```

```
        cout << root->data << " ";
```

```
        inorder(root->right);
```

```
    }
```

```
}
```

```
int main() {
```

```
    Node* root = NULL;
```

```
    root = insertIterative(root, 12);
```

```
    insertIterative(root, 8);
```

```
    insertIterative(root, 18);
```

```

insertIterative (root, 5);
insertIterative (root, 11);
insertIterative (root, 17);
insertIterative (root, 4);

```

```

insertIterative (root, 10);

```

```

cout << "Inorden Traversal: ";

```

```

inorder (root);

```

```

return 0;

```

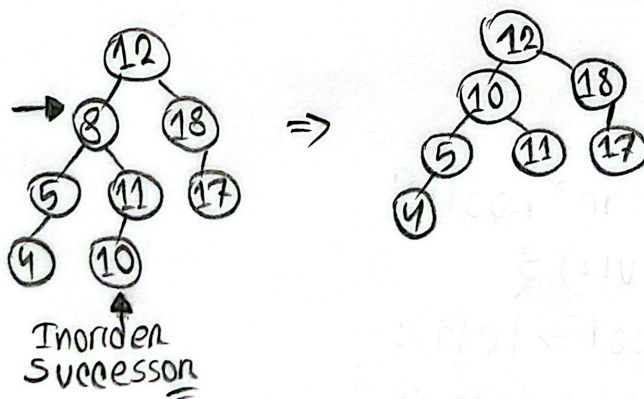
```

}

```

(vii) Deleting node 8:

• Hand:



• Code:

```

#include <iostream>
using namespace std;

```

```

struct Node {
    int data;
    Node *left;
    Node *right;
}

```

```

Node (int value) {
    data = value;
    left = right = NULL;
}
};

```

```

Node* findMin (Node* root) {
    while (root && root->left != NULL)
        root = root->left;
    return root;
}

```

```

Node* deleteNode (Node* root, int key) {
    Node* parent = NULL;
    Node* curr = root;

    while (curr && curr->data != key) {
        parent = curr;
        if (key < curr->data)
            curr = curr->left;
        else
            curr = curr->right;
    }

    if (!curr)
        return root;

    if (curr->left == NULL || curr->right == NULL) {
        Node* newCurr;
        if (curr->left == NULL)
            newCurr = curr->right;
    }
}

```

else

newCurr = curr → left;

if (parent == NULL)

return newCurr;

if (curr == parent → left)

parent → left = newCurr;

else

parent → right = newCurr;

delete curr;

}

else {

Node* successor = findMin(curr → right);

int value = successor → data;

root = deleteNode(root, successor → data);

curr → data = val;

}

return root;

}

void inorder(Node* root) {

if (root) {

inorder(root → left);

cout << root → data << " ",

inorder(root → right);

}

}

```
int main() {
```

```
Node* root = NULL;
```

```
int arr[] = {12, 8, 18, 5, 11, 17, 4, 10};
```

```
for (int x: arr)
```

```
    root = insertIterative(root, x);
```

```
root = deleteNode(root, 8);
```

```
cout << "After Deletion: ";
```

```
inorder(root);
```

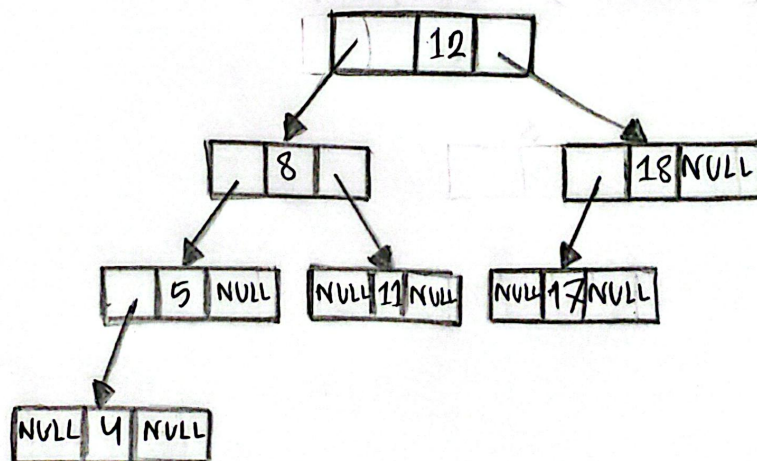
```
return 0;
```

```
}
```

viii) Memory Representation:

12	8	18	5	11	17	-	4	-	-	-
0	1	2	3	4	5	6	7	8	9	10

Linked List Representation:



Left Child Address	Data	Right Child Address
--------------------------	------	---------------------------

(ix) Tree Traversal Using Stack:

□ Preorder:

Stack Operation

Output

(i) push 12

(ii) pop 12 → print 12

(iii) push 18, push 8

(iv) pop 8 → print 8

(v) push 11, push 5

(vi) pop 5 → print 5

(vii) push 4

(viii) pop 4 → print 4

(ix) pop 11 → print 11

~~(x) push 10~~

~~(xi) pop 10 → print 10~~

(xii) pop 18 → print 18

(xiii) pop 17 → print 17

12

~~12~~ 12 8

12 8 5

12 8 5 4

12 8 5 4 11

~~12 8 5 4 11 10~~

12 8 5 4 11 18 ~~18~~

12 8 5 4 11 18 ~~18~~ 17

□ Preorder ⇒ 12, 18, 5, 4, 11, 18, 17

□ Inorder:

Stack Operation

Output

(i) push 12

(ii) push 8

(iii) push 5

(iv) push 4

(v) pop 4 → print 4

(vi) pop 5 → print 5

(vii) pop 8 → print 8

(viii) push 11

(ix) pop 11 → print 11

(x) pop 12 → print 12

(xi) push 18

(xii) push 17

(xiii) pop 17 → print 17

(xiv) pop 18 → print 18

(xv)

4

4 5

4 5 8

4 5 8 11

4 5 8 11 12

4 5 8 11 12 17

4 5 8 11 12 17 18

□ Inorder ⇒ 4, 5, 8, 11, 12, 17, 18

□ Postorder:

Stack Operation (stack 1)

Stack 2

- | | |
|---|---|
| (i) push 12 | |
| (ii) pop 12 → push S2 | 12 |
| (iii) push 8, 18 | |
| (iv) pop 18 → push S2 | 12, 18 |
| (v) push 11 17 | |
| (vi) pop 17 → push S2 | 12, 18, 17 |
| (vii) pop 8 → push S2 | 12, 18, 17, 8 |
| (viii) pop 11 → push 11 5, 11 | 12, 18, 17 , 17, 8 |
| (ix) pop 11 → push S2 | 12, 18, 17 , 8, 11 |
| (x) pop 5 → push S2 | 12, 18, 17, 8, 11, 18 5 |
| (xi) pop 11 → push 11 4 | 12, 18, 17, 8, 11, 18 5 |
| (xii) pop 4 → push S2 | 12, 18, 17, 8, 11, 5, 4 |

□ To get the final output result,
we pop all elements from stack 2...

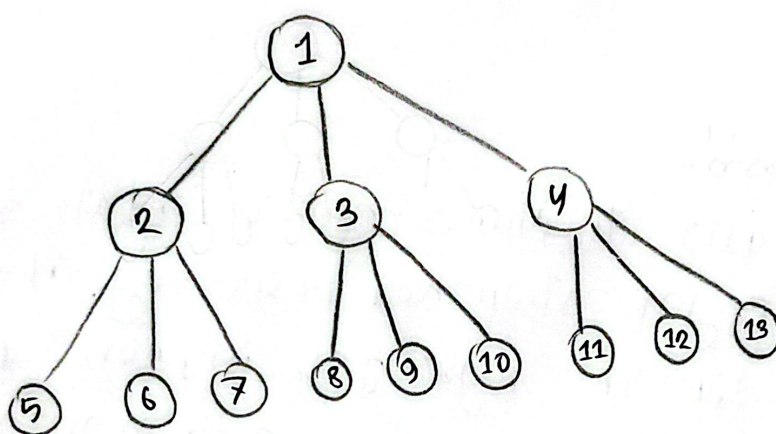
∴ Postorder ⇒ 4, 5, 11, 8, 17, 18, 12 //

(x) m-ary tree:

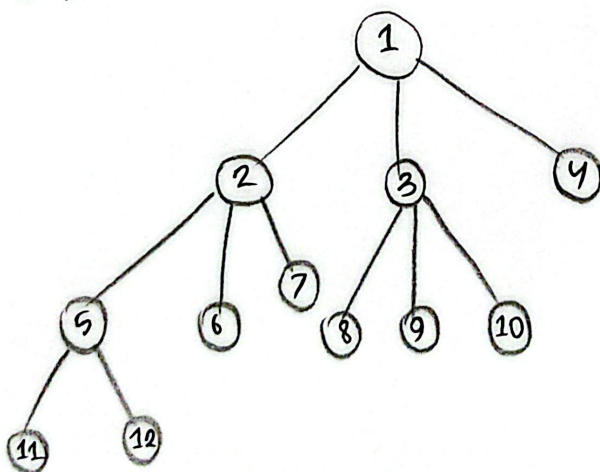
An m-ary tree is a rooted tree where each node has no more than m children

Considering, $m=3$;

□ 3-ary tree:



or,



The above two trees are the examples of m-ary tree where $m=3$. Basically, in this type of tree, each node can have children upto m ($m=3$ for this case). If any node have more than m children then the condition will break and it will no longer be called m-ary tree; m-ary trees do not require ordering constraints.

■ Reflection:

This assignment improved my understanding of different tree structures and their real-life applications. I learnt how operations like insertion, deletion and traversal work internally and how stack based methods simulates ~~in~~ inorder, preorder and post order traversals.

■ Conclusion:

Tree data structures are essentials for efficient data organization and processing. Choosing the right tree for the right job improves performance in searching, ~~ex~~ sorting and managing data in real systems.